CHAPTER 2 ■ COMMAND REFERENCE

Linux System Architecture, Command Line & File Management

Principles of Linux Operating Systems | Weeks 3–4

English explanations • Ubuntu terminal examples • Username: updikhadir

Opening terminal and moving around

Ubuntu terminal | updikhadir practice

```
updikhadir@ubuntu:~$ pwd
/home/updikhadir
updikhadir@ubuntu:~$ cd /etc
updikhadir@ubuntu:/etc$ cd ..
updikhadir@ubuntu:/$ ls
bin boot dev etc home proc sys tmp usr var
updikhadir@ubuntu:/$ cd ~
updikhadir@ubuntu:~$
```

Chapter objectives

Understand Ubuntu system layers and filesystem directories; use Bash and terminal features; read command syntax; find help; use quotes, escapes, wildcards, variables and PATH; navigate directories; and create, copy, move and remove files safely.

Terminal username: All terminal examples in this PDF use `updikhadir` instead of the original slide username `student`. Example outputs are illustrative and may vary on your Ubuntu installation.

2.1 ■ Ubuntu System Architecture

Definition

A Linux system is built in layers, from physical hardware up to the users. Each layer communicates with the layer next to it. The kernel manages the hardware, while the shell and utilities let users request work.

Why students use this section

Understanding the layers helps explain what happens after you enter a command and where a virtual machine fits.

Practice screenshot — test command and example output

```
● ● ● Ubuntu terminal | updikhadir practice

updikhadir@ubuntu:~$ ls /home
updikhadir

# Command path:
User -> Shell (Bash) -> Utility (ls) -> Kernel -> Hardware
Hardware result returns through the same layers.
```

Note: The six layers shown in the chapter are hardware, kernel, shell, system utilities, applications and users. GNOME is optional and belongs with the applications layer.

Six layers of an Ubuntu system

Layer	What it does	Examples from Chapter 2
Hardware	Physical resources.	CPU, RAM, disks, network card
Kernel	Manages hardware and system resources.	CPU time, memory, devices, files
Shell	Reads commands and starts programs.	Bash
System utilities	Programs that perform system tasks.	ls, cp, mkdir in /usr/bin
Applications	Programs used by people.	Firefox, LibreOffice, Files
Users	People and accounts using the system.	You and other user accounts

Host, guest and hypervisor

Host: the real computer and the operating system started first. **Hypervisor:** software such as VirtualBox that creates virtual machines and shares host resources. **Guest:** Ubuntu running inside the VM with its own kernel and virtual hardware.

```
● ● ● Ubuntu terminal | updikhadir practice

HOST: physical computer + host operating system
|
+-- Hypervisor: VirtualBox
|
+-- GUEST: Ubuntu VM
Apps -> Bash -> Utilities -> Guest kernel
Virtual CPU, RAM and virtual disk
```

2.2 ■ The Filesystem Hierarchy

Definition

Linux organises files in one directory tree beginning at the root directory `/`. Unlike Windows drive letters, disks and USB devices appear somewhere within this single tree.

Main directories

Directory	What it contains	Example / reminder
/	Root of the whole filesystem.	Everything starts here.
/home	Users' personal files.	/home/updikhadir
/etc	System-wide configuration files.	/etc/hostname
/var	Changing data such as logs, caches and queues.	Check here when disk space fills.

Directory	What it contains	Example / reminder
/usr	Installed programs and their files.	/usr/bin
/tmp	Temporary files; usually cleared at reboot.	Scratch space.
/boot	Kernel and files needed to start the system.	Boot files.
/dev	Device files for disks, terminals and USB.	/dev/sda
/proc	Live process and kernel information.	/proc/cpuinfo, /proc/meminfo
/sys	Live device and driver information.	/sys/class/net

Ubuntu terminal | updikhadir practice

```
updikhadir@ubuntu:~$ ls /
bin boot dev etc home lib media
mnt opt proc root run sbin snap
srv sys tmp usr var
updikhadir@ubuntu:~$ ls /sys/class/net
enp0s3 lo
```

Important: `/proc` and `/sys` are pseudo-filesystems built by the kernel in memory. They are not ordinary files stored on disk and disappear at shutdown.

2.3 ■ The Terminal and Bash

Definition

The terminal window and the shell inside it are two different programs. GNOME Terminal displays text and passes keyboard input. Bash reads the command line and starts programs. The kernel performs the low-level work.

Why students use this section

Students use terminal history and tab completion to work faster and avoid typing mistakes.

Practice screenshot — test command and example output

```
Ubuntu terminal | updikhadir practice

updikhadir@ubuntu:~$ echo $SHELL
/bin/bash
updikhadir@ubuntu:~$ history
1 pwd
2 ls
3 cd /etc
4 cd ~
5 history
```

Note: Other shells exist, including zsh, dash and ksh. Ubuntu uses Bash by default in the course environment.

History shortcuts

Shortcut / file	Function	How to use it
Up / Down arrows	Move backward or forward through earlier commands.	Edit the recalled line and press Enter.
history	Prints a numbered list of commands run in the shell.	Type history.
Ctrl + R	Searches backward through command history.	Type part of a previous command.
~/.bash_history	Stores command history when Bash exits.	Hidden file in the home directory.
Tab	Completes a matching command or filename.	Press once after typing the beginning.
Tab Tab	Lists available matches when there is more than one.	Type another letter to narrow choices.

```
Ubuntu terminal | updikhadir practice

updikhadir@ubuntu:~$ cd Doc<Tab>
updikhadir@ubuntu:~$ cd Documents/

updikhadir@ubuntu:~$ cd D<Tab><Tab>
Desktop/ Documents/ Downloads/

updikhadir@ubuntu:~$ mkd<Tab>
updikhadir@ubuntu:~$ mkdir
```

Remember: If Tab does not complete a name, check spelling and capital letters. Bash can complete both filenames and program names found through PATH.

2.4 ■ Command Syntax

Definition

Most command lines follow the pattern: command, options and arguments. The command names the program; options change its behaviour; arguments identify what the command works on.

Why students use this section

Breaking a command into these parts helps you understand unfamiliar instructions and predict what they will do.

Syntax / form

```
command [OPTIONS] [ARGUMENTS]
```

Practice screenshot — test command and example output

Ubuntu terminal | updikhadir practice

```
updikhadir@ubuntu:~$ ls -l /home
COMMAND: ls
OPTION: -l
ARGUMENT: /home
```

Note: Only the command is required. Options and arguments are optional depending on the command.

Option form	Meaning	Example
-a	Short option: one dash and one letter.	ls -a
-l	Long listing format.	ls -l
-la	Combine short options.	ls -la
--all	Long option: two dashes and a word.	ls --all
-h / --human-readable	Show sizes using K, M or G.	ls -lh
-r / --recursive	Work through subdirectories (for cp).	cp -r DIR NEWDIR
-i / --interactive	Ask before overwriting or deleting.	rm -i FILE

Watch the dashes: `-all` is not the same as `--all`. One dash introduces short letters; two dashes introduce a long option.

2.5 ■ Getting Help: man, --help and info

Definition

Ubuntu includes built-in documentation. The man command opens a full manual page, --help prints a short summary, and info opens longer chapter-style documentation, mostly for GNU tools.

Why students use this section

Use help tools to check options on the installed system instead of relying on memory or possibly outdated web instructions.

Practice screenshot — test command and example output

```
Ubuntu terminal | updikhadir practice
updikhadir@ubuntu:~$ man ls
NAME
ls - list directory contents
SYNOPSIS
ls [OPTION]... [FILE]...

Press q to leave the manual page.
updikhadir@ubuntu:~$ mkdir --help
Usage: mkdir [OPTION]... DIRECTORY...
-p, --parents make parent directories as needed
```

Command	Purpose	When to use it
man COMMAND	Opens the full manual page.	Check exactly what an option does.
COMMAND --help	Prints a short summary on screen.	Quick reminder of an option.
info COMMAND	Opens longer GNU documentation.	Learn a tool in more depth.
help COMMAND	Shows help for Bash built-ins.	Use for cd, which is part of Bash.
man 1 passwd	Manual section 1: user command.	Read about the passwd command.
man 5 passwd	Manual section 5: file format.	Read about the passwd file.

Useful keys inside man

Key	Action	Description
Space	Next page	Move forward.
b	Back a page	Move backward.
/word	Search	Find a word in the manual.
n	Next match	Move to the next search result.
q	Quit	Return to the terminal.

```
Ubuntu terminal | updikhadir practice
updikhadir@ubuntu:~$ man rmdir
# Read the NAME, SYNOPSIS and DESCRIPTION sections
# Press q to quit
updikhadir@ubuntu:~$ help cd
# Bash displays help for the cd built-in
```

Common trap: `man cd` **may report no manual entry because cd is a Bash built-in. Use** `help cd`**.**

2.6 ■ Quoting, Escaping and Wildcards

Definition

Bash interprets special characters before passing a command to a program. Spaces separate words, $ expands a variable, and wildcard characters can expand into matching filenames. Quotes and backslashes control this behaviour.

Why students use this section

Use quoting and escaping to pass filenames with spaces or special characters correctly. Use wildcards to work with groups of files.

Practice screenshot — test command and example output

```
Ubuntu terminal | updikhadir practice

updikhadir@ubuntu:~/lab$ echo Hello $USER
Hello updikhadir
updikhadir@ubuntu:~/lab$ echo "Hello $USER"
Hello updikhadir
updikhadir@ubuntu:~/lab$ echo 'Hello $USER'
Hello $USER

updikhadir@ubuntu:~/lab$ touch "new file"
updikhadir@ubuntu:~/lab$ ls
'new file'
```

Quotes and escaping

Form	What Bash does	Example
"double quotes"	Keeps spaces together but still expands $ variables.	echo "Hello $USER"
'single quotes'	Keeps the enclosed text exactly as typed.	echo '$HOME'
\ backslash	Protects the one special character immediately after it.	echo \$HOME
Quotes around filename	Makes a spaced filename one argument.	touch "week 3.txt"
Escaped space	Protects a space in a filename.	cd lab\ report

Wildcards expanded by Bash

Wildcard	Matches	Example
*	Any characters, or none.	ls *.txt
?	Exactly one character.	ls lab?.txt
[abc]	One character from the listed set.	ls lab[12].txt
[0-9]	One character in the range.	ls *[0-9].jpg

Ubuntu terminal | updikhadir practice

```
updikhadir@ubuntu:~/lab$ ls *.txt
lab1.txt lab10.txt lab2.txt notes.txt

updikhadir@ubuntu:~/lab$ ls lab?.txt
lab1.txt lab2.txt

updikhadir@ubuntu:~/lab$ ls lab[12].txt
lab1.txt lab2.txt
```

Safety reminder: Run `ls` with a wildcard pattern before using `rm` with that pattern. The * wildcard does not match hidden names by default.

2.7 ■ Variables, PATH and ~/.bashrc

Definition

Bash stores information in variables such as the username, home directory and PATH. NAME=value creates a shell variable; $NAME reads its value. export passes a variable to programs started from the current shell.

Why students use this section

PATH tells Bash which directories to search when you type a program name. The ~/.bashrc file stores personal Bash settings for new terminals.

Practice screenshot — test command and example output

Ubuntu terminal | updikhadir practice

```
updikhadir@ubuntu:~$ COURSE=linux
updikhadir@ubuntu:~$ echo $COURSE
linux
updikhadir@ubuntu:~$ echo $HOME $USER
/home/updikhadir updikhadir
updikhadir@ubuntu:~$ export COURSE
updikhadir@ubuntu:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:...
```

Command / variable	Purpose	Example
NAME=value	Creates or changes a shell variable; no spaces around =.	COURSE=linux
$NAME	Reads the value of a variable.	echo $COURSE
export NAME	Exports a variable to child programs.	export COURSE
$HOME	Shows the current user's home directory.	echo $HOME
$USER	Shows the current username.	echo $USER
$PATH	Shows directories searched for commands.	echo $PATH
~/.bashrc	Personal Bash configuration file.	ls -a ~

How PATH works

PATH is a colon-separated list of directories. Bash checks those directories from left to right and runs the first matching program. If no match is found, the shell reports command not found. The current directory is not searched unless it is included in PATH.

Ubuntu terminal | updikhadir practice

```
updikhadir@ubuntu:~$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:...
updikhadir@ubuntu:~$ ls /usr/bin/ls
/usr/bin/ls
updikhadir@ubuntu:~$ ls -a ~
. .. .bash_history .bashrc .profile
Desktop Documents Downloads ...
```

Important: A variable typed at the prompt lasts for the current shell session. A setting written in ~/.bashrc is applied when new interactive Bash terminals start.

2.8 ■ Paths and Filenames

Definition

A path is an address in the filesystem tree. An absolute path starts at / and has the same meaning wherever you are. A relative path starts from the current directory.

Why students use this section

Correct paths are essential for navigating and managing files. Linux filenames are case-sensitive, and names beginning with a dot are hidden from a normal ls listing.

Practice screenshot — test command and example output

```
Ubuntu terminal | updikhadir practice

updikhadir@ubuntu:~/Documents$ cd /home/updikhadir/Downloads
updikhadir@ubuntu:~/Downloads$ cd ../Documents
updikhadir@ubuntu:~/Documents$ cd ..
updikhadir@ubuntu:~$ cd ~/Documents
updikhadir@ubuntu:~/Documents$ pwd
/home/updikhadir/Documents
```

Path form	Meaning	Example
Absolute path	Starts at root /; same meaning from any directory.	/home/updikhadir/Downloads
Relative path	Starts from the current directory.	../Downloads
.	Current directory.	cp /etc/hostname .
..	Parent directory, one level up.	cd ..
~	Current user's home directory.	cd ~/Documents

Filename rules from Chapter 2

Rule	Explanation	Example
Case matters	Uppercase and lowercase names are different.	notes.txt ≠ Notes.txt
Leading dot	A name beginning with . is hidden in normal ls output.	.bashrc
Extensions	Extensions are labels, not a Linux requirement.	report.pdf
/ character	Not allowed inside a filename; it separates directories.	Use a different character.
Empty name	A filename cannot be empty.	Give the file a name.
Length	Names can be up to 255 bytes.	Keep names sensible.

Recommended names: Use letters, digits, hyphen (-), underscore (_) and dot (.). Avoid spaces and symbols where possible, and do not start a name with a hyphen because commands may interpret it as an option.

2.9 ■ Navigating with pwd, cd and ls

Definition

pwd prints the current directory, cd changes the current directory, and ls lists the contents of a directory.

Why students use this section

These three commands are the basic tools for moving around the Linux filesystem and checking where you are.

Practice screenshot — test command and example output

Ubuntu terminal | updikhadir practice

```
updikhadir@ubuntu:~$ pwd
/home/updikhadir
updikhadir@ubuntu:~$ cd /var/log
updikhadir@ubuntu:/var/log$ cd ..
updikhadir@ubuntu:/var$ cd -
/var/log
updikhadir@ubuntu:/var/log$ cd
updikhadir@ubuntu:~$
```

Command	What it does	Example
pwd	Prints current directory as an absolute path.	pwd
cd /var/log	Moves to an absolute path.	cd /var/log
cd Documents	Moves to a relative directory below here.	cd Documents
cd ..	Moves one level up to the parent.	cd ..
cd or cd ~	Moves to the home directory.	cd
cd -	Returns to the previous directory and prints it.	cd -
ls	Lists names in the current directory.	ls
ls -a	Includes hidden names and . and ..	ls -a
ls -l	Long listing, one line per entry.	ls -l
ls -lh	Long listing with human-readable sizes.	ls -lh

Ubuntu terminal | updikhadir practice

```
updikhadir@ubuntu:~$ ls -lh
total 2.3M
drwxr-xr-x 2 updikhadir updikhadir 4.0K Sep 14 10:02 Documents
-rw-rw-r-- 1 updikhadir updikhadir 12K Sep 14 10:05 notes.txt
-rw-rw-r-- 1 updikhadir updikhadir 2.3M Sep 14 10:07 slides.pdf
```

Reading ls -lh: the first character shows the type: d means directory and - means regular file. The listing also displays permissions, links, owner, group, size, modification time and name. Permissions are studied in Chapter 5.

2.10 ■ Creating and Managing Files

Definition

Linux provides commands to create empty files and directories, copy files, move or rename them, and remove files and directories. The rm command has no recycle bin.

Why students use this section

Use these commands to organise project work. Check the current directory and filenames before copying, moving or deleting anything.

Practice screenshot — test command and example output

```
Ubuntu terminal | updikhadir practice

updikhadir@ubuntu:~/lab$ touch notes.txt todo.md
updikhadir@ubuntu:~/lab$ mkdir week3
updikhadir@ubuntu:~/lab$ mkdir -p projects/linux
updikhadir@ubuntu:~/lab$ rmdir week3
updikhadir@ubuntu:~/lab$ ls
notes.txt todo.md projects
```

Command	Purpose	Example
touch FILE…	Creates empty files or updates timestamps of existing files.	touch notes.txt todo.md
mkdir DIR…	Creates directories.	mkdir week3
mkdir -p DIR…	Creates missing parent directories as needed.	mkdir -p projects/linux
rmdir DIR…	Removes empty directories only.	rmdir week3
cp SOURCE DEST	Copies a file.	cp notes.txt notes.bak
cp -r DIR NEWDIR	Copies a directory and its contents recursively.	cp -r projects projects-copy
mv OLD NEW	Moves or renames a file or directory.	mv draft.txt final.txt
rm FILE…	Removes files; no recycle bin.	rm notes.bak
rm -i FILE…	Asks before removing each file.	rm -i notes.bak
rm -r DIR	Removes a directory and everything inside it.	rm -r old-project

Copying and moving: the difference

Command	Before	After
cp notes.txt notes.bak	notes.txt	notes.txt and notes.bak
mv notes.txt old.txt	notes.txt	old.txt
cp notes.txt backup/	notes.txt and backup/	Original plus a copy inside backup/
mv final.txt ~/Documents/	final.txt in current directory	File moved into Documents

```
Ubuntu terminal | updikhadir practice

updikhadir@ubuntu:~/lab$ cp notes.txt notes.bak
updikhadir@ubuntu:~/lab$ cp -r projects projects-copy
updikhadir@ubuntu:~/lab$ mv draft.txt final.txt
updikhadir@ubuntu:~/lab$ rm -i notes.bak
rm: remove regular empty file 'notes.bak'? n
```

Critical safety warning: rm removes files without moving them to Trash. Use -i while learning. The chapter says to avoid `rm -rf`; it removes recursively without questions or errors. Before pressing Enter, check with pwd and ls.

2.11 ■ Lab: Navigating and Managing Files

Purpose

Practise the Chapter 2 commands in a fresh directory called ~/ch2lab. Type the commands yourself and check the results as you go.

Part A — build and navigate

Step	Type this	What it should do
1	cd	Takes you home.
2	mkdir -p ~/ch2lab/notes	Creates ch2lab and notes.
3	mkdir ~/ch2lab/backup	Creates backup beside notes.
4	cd ~/ch2lab	Moves into the lab folder; check with pwd.
5	touch notes/day1.txt notes/day2.txt "notes/week 3.txt"	Creates three empty files; quotes keep week 3 as one name.
6	ls -l notes	Shows a long listing of the three files.
7	ls -a ~	Lists hidden names in your home; find .bashrc.

Part B — copy, move, remove and look things up

Step	Type this	What it should do
8	cp notes/day1.txt backup/	Copies day1.txt into backup.
9	mv notes/day2.txt notes/monday.txt	Renames day2.txt.
10	cp -r notes notes-copy	Copies the whole notes directory.
11	rm -i notes-copy/*	Asks before removing each matching file.
12	rmdir notes-copy	Removes the directory after it is empty.
13	echo $PATH	Shows the program search path; find /usr/bin.
14	man rmdir	Read the manual and find the option for empty parent directories; press q to quit.

Lab safety: Work only inside ~/ch2lab. If rmdir reports that a directory is not empty, inspect it with ls -a before continuing.

Command reference table

Command / concept	Short explanation	Section
pwd	Print current working directory.	2.9
cd, cd -, cd ~	Change directory, return to previous, or go home.	2.9
ls, ls -a, ls -l, ls -lh	List files, including hidden names and details.	2.9
history	Show previously run commands.	2.3
man, --help, info, help	Read built-in command documentation.	2.5
touch	Create empty files / update timestamps.	2.10
mkdir, mkdir -p	Create directories, including parent directories.	2.10
rmdir	Remove empty directories only.	2.10
cp, cp -r	Copy files or directories.	2.10
mv	Move or rename files and directories.	2.10
rm, rm -i, rm -r	Remove files/directories; use interactive mode carefully.	2.10
echo $NAME	Display a variable value.	2.7
export NAME	Pass a variable to child programs.	2.7
echo $PATH	Display directories searched for programs.	2.7
Quotes, \, *, ?, [ ]	Control text and filename matching in Bash.	2.6

Key points to remember

- Linux has one filesystem tree starting at /; there are no drive letters.
- The terminal is a window; Bash is the shell that reads commands.
- A command line contains a command, optional options and optional arguments.
- Use man, --help, info or help to look up unfamiliar commands.
- Double quotes preserve spaces but still expand variables; single quotes preserve text exactly.
- PATH is a colon-separated list of directories searched by Bash.
- Absolute paths start at /; relative paths start from the current directory.
- Check your location with pwd before moving or deleting files.
- rm has no Trash; use -i while learning and keep backups.

Source note

This study guide is based on the supplied file “Chapter2 Linux System Architecture and Management.pptx” (Chapter 2, Principles of Linux Operating Systems). The explanations, section order, terminology and examples follow the slides. Terminal prompts use the requested username updikhadir in place of student. Example output is illustrative and may differ by Ubuntu version.